TEAM CONTRACT

1.Team information

Tên nhóm:
Tên đồ án:

Student ID	Full name	Email	Communication method
25127031	Nguyễn Hoàng Danh	Phamthanhthuy7797@gmail.com	Zalo,Facebook
25127527	Lê Minh Trí	letri0107@gmail.com	Zalo, Facebook
25127397	Nguyễn Trung Kiên	nkien3142@gmail.com	Zalo, Facebook
25127467	Trương Tấn Phúc	ttphuc2531@clc.fitus.edu.vn	Zalo, Facebook
25127034	Phạm Tiến Đạt	phamtiendat37@gmail.com	Zalo, Facebook
25127304	Hồ Hoàng Tiến Dũng	hodung300107@gmail.com	Zalo, Facebook
25127549	Nguyễn Chí Vĩ	vinguyen.14200@gmail.com	Zalo, Facebook

2.Team Goals and Expectations

Project Objectives: The goal is to turn an ambiguous human need into a reliable computational process: understand → decompose → represent → decide → execute → verify → improve.
Target grade or quality level that the team aims to achieve: Achieve a high grade of around 8–9 points. More importantly, ensure that every member develops their thinking skills equally and gains an understanding of computational thinking.
The level of effort and time that each member commits to contributing: Commit to actively contributing to the project by completing assigned tasks on time, attending team meetings regularly, and dedicating sufficient time each week to ensure the quality of the work.

3.Roles and Responsibilities

Nguyễn Hoàng Danh:

- Role: Team Leader, Code Reviewer, Presenter, Documentation
- Strenths: Has good knowledge of Git, including branch management and resolving merge conflicts, has C++ programming skills, careful, with the ability to test and identify bugs.
- Weaknesses: Has limited experience in user interface design, has limited experience in integrating system components

Lê Minh Trí:

- Role: Feature Developer, Code reviewer, Presenter
- Strengths: Good logical thinking, basic understanding of programming algorithms and eager to learn.
- Weaknesses: Limited experience with advanced Git commands and a bit quiet.

Nguyễn Trung Kiên:

- Role: Feature Developer, Presenter
- Strengths: Good logical thinking, basic understanding of programming algorithms and eager to learn.
- Weaknesses: Limited experience with advanced Git commands and a bit quiet.

Trương Tấn Phúc

- Role: Feature Developer, Presenter
- Strengths: A solid foundation in C++, has ability to think logically, and a high sense of responsibility.
- Weaknesses: Limited practical programming experience as well as a tendency to get distracted when doing multiple tasks at once.

Phạm Tiến Đạt:

- Role: Feature Developer, Presenter
- Strengths: Strong logical reasoning, solid foundation in C++ , proactive problem solver with high sense of responsibility.
- Weaknesses: Needs to be more cautious when managing Git branching workflows and resolving complex merge conflicts.

Hồ Hoàng Tiến Dũng

- Role: Feature Developer, Presenter
- Strengths: A solid foundation in C++, has ability to think logically, and a high sense of responsibility.
- Weaknesses: Limited practical programming experience as well as a tendency to get distracted when doing multiple tasks at once.

Nguyễn Chí Vĩ:

- Role: Feature Developer, Presenter
- Strenths: Efficiently managing Git/GitHub workflows, branches, and resolving team code conflicts, Strong sense of responsibility in teamwork and a high capacity for self-learning.
- Weaknesses: Adapting from C++ resource management to the high-level, Python-dominated AI ecosystem, Need to quickly learn specialized agent tools.

4.Communication

- Communication Channels: Zalo, facebook,discord
- Expected message response time: Around 1 to 2 hours
- The person primarily responsible for communicating with the lecture: Nguyễn Hoàng Danh (Leader)

5.Meetings

5.1. Frequency and Meeting Schedule

- The team will hold one regular meeting each week to review progress, discuss issues, and plan upcoming tasks.
- Additional meetings may be held when necessary, especially before internal deadlines or important project milestones.
- Meetings may be held in person or online, depending on members' schedules and the team's agreement.

5.2. Meeting Agenda

- The meeting organizer will prepare and share a brief agenda before each meeting.
- Each member should report completed tasks, current progress, upcoming tasks, and any obstacles encountered.
- The team will prioritize issues that affect project progress, dependencies between tasks, or completion deadlines.

5.3. Meeting Minutes

- An assigned member will record the meeting minutes, including decisions, assigned tasks, responsible members, and deadlines.
- Meeting minutes will be stored in the shared repository or the team's document-storage space so that all members can access them.
- The responsibility for taking meeting minutes may rotate among team members.

5.4. Punctuality and Attendance

- Members are expected to arrive on time and participate actively in meetings.
- Members who expect to be late or absent must inform the team through the agreed communication channel as early as possible.
- Members who cannot attend must read the meeting minutes and continue carrying out their assigned responsibilities. Each member may have at most two excused absences.

5.5. Meeting Effectiveness

- Each meeting must have clear objectives and end with a summary of decisions and follow-up actions.
- The team should avoid unnecessary meetings and use asynchronous communication for simple updates when appropriate.

6. Workflow and Standards

6.1. Task Assignment and Tracking

- The team will use GitHub Projects and GitHub Issues to assign and track work.
- Each task must have a clear description, a primary assignee, an expected deliverable, and an internal deadline.
- Tasks should be divided into manageable parts that can be checked and evaluated independently.
- Each member is responsible for updating the status of their tasks and informing the team when obstacles arise.
- The primary assignee may still receive support or feedback from other members.

6.2. Internal Deadlines

- When possible, internal deadlines should be earlier than official submission deadlines to allow time for integration, testing, documentation, and bug fixes.
- Members must inform the team immediately if they identify a risk of missing a deadline, explain the reason, and propose a recovery plan.
- Any deadline changes must be communicated and agreed upon by affected members and the team.

6.3. Git Workflow

- The team will maintain a shared private repository for the project.
- The `main` branch will contain the stable version of the project. Contributors must not push directly to this branch.
- Members must create a separate branch for each task or feature and use descriptive names, such as `feature/data-processing`, `fix/input-validation`, or `docs/user-guide`.
- Before integration, members should synchronize their branch with relevant recent changes and carefully resolve conflicts.
- Commits must be reasonably scoped, understandable, and related to a specific change.
- Commit messages must follow a consistent format, such as `feat: add budget calculation` or `fix: handle invalid input`.
- After completing work, members must push their branch to the remote repository and create a Pull Request (PR) targeting `main`.
- Two designated members will review Pull Requests. The PR author may not approve their own PR. Members who are not assigned as reviewers may provide feedback through GitHub Issues but may not approve the PR.
- Each PR must clearly describe its changes, link to relevant tasks when applicable, and include testing evidence.

6.4. Code Review and Integration

- Reviewers must check correctness, readability, consistency with project requirements, potential edge cases, and relevant test coverage.
- Review comments must be constructive, specific, and focused on improving the product rather than criticizing the author.
- The PR author is responsible for addressing feedback or explaining why a proposed change is unnecessary.
- A PR may be merged only after it meets review requirements and all applicable checks have passed.
- Integration conflicts and failing tests must be resolved before related changes are considered complete.

6.5. Source Code Quality Standards

- Source code must be readable, consistently formatted, and organized into appropriate modules or functions.
- Variable names, function names, and file names must be meaningful and used consistently.
- Important or difficult-to-understand logic should include brief comments when necessary.
- Input validation, error handling, and relevant edge cases must be considered.
- Each feature must be tested before being submitted for review.
- New changes must not intentionally break existing functionality.

6.6. Documentation Standards

- The project must maintain an up-to-date `README.md` describing the project, its purpose, installation instructions, usage instructions, and relevant limitations.
- Technical documentation must accurately reflect the implemented system.
- External sources, datasets, libraries, and AI-assisted materials must be acknowledged when necessary.

- Documentation must be reviewed and updated when significant changes affect system behavior, installation, or usage.

6.7. Definition of Done

A task is considered complete only when:

1. The agreed requirements have been implemented.
2. Relevant tests have been performed and identified defects have been resolved.
3. The source code or documentation has been reviewed when necessary.
4. Relevant documentation has been updated.
5. The changes have been integrated into the appropriate branch and verified.

7. Decision-Making

7.1. Decision-Making Principles

- The team prioritizes open discussion and consensus for important decisions involving project scope, system architecture, task assignments, and deliverable quality.
- Members are encouraged to present differing opinions and provide reasons or evidence to support their proposals.
- Technical decisions within an assigned task may normally be made by the responsible member, provided that they comply with project requirements and agreed technical standards.

7.2. Voting Procedure

- If consensus cannot be reached after reasonable discussion, the team will vote by simple majority among current members.
- Each member has one vote.
- A proposal is approved when it receives more than half of the valid votes cast by members eligible to participate in the decision.
- Members unable to attend a decision-making meeting may submit their opinion or vote through the agreed communication channel before the specified deadline.
- Decisions involving significant changes to project scope, workload distribution, or the team agreement must be recorded.

7.3. Tie-Breaking and Escalation

- If disagreement remains unresolved, the team leader will facilitate a final discussion and propose a practical solution.
- For decisions that significantly affect academic requirements, technical feasibility, or fairness, the team should consult the lecturer when necessary.
- The team leader does not have unlimited authority to override agreed decisions or disregard the team agreement.

8. Conflict Resolution

8.1. Direct Discussion

- Members should first discuss disagreements directly and respectfully with the people involved.

- Discussion should focus on facts, project requirements, responsibilities, and feasible solutions rather than personal criticism.
- Members should try to understand different perspectives before reaching a conclusion.

8.2. Team Leader Facilitation

- If the issue cannot be resolved directly, the team leader will facilitate a discussion among the relevant members.
- The team leader must listen to relevant viewpoints, clarify responsibilities, and help find a fair and practical solution.
- When appropriate, agreed actions, responsible members, and deadlines must be recorded.

8.3. Full-Team Discussion

- If a conflict affects the whole project, task distribution, or overall progress, it may be discussed by the whole team and resolved according to Section 7.

8.4. Requesting Lecturer Support

- If a conflict remains unresolved or involves serious issues such as repeated non-participation, unfair treatment, academic-integrity violations, or prolonged failure to fulfill responsibilities, the team may ask the lecturer for guidance.
- Relevant evidence, including task history, meeting minutes, and repository activity, must be presented objectively.
- Members must not retaliate against, harass, or unfairly disadvantage anyone who raises a concern in good faith.

8.5. Resolution and Follow-Up

- The team must record the agreed solution, assign follow-up actions when necessary, and assess whether the solution is effective.
- All members are responsible for cooperating to implement the agreed solution.

9. Responsibilities and Handling Violations

9.1. Individual Responsibilities

- Each member is responsible for completing assigned tasks according to the agreed requirements and deadlines.
- Members must update progress honestly and promptly report obstacles or potential delays.
- Each member must contribute to agreed activities such as planning, implementation, testing, documentation, review, or other project work.
- Members must not claim credit for work they did not perform or deliberately misrepresent their level of contribution.

9.2. Conduct That May Violate the Agreement

Possible violations include:

- Repeatedly failing to complete tasks on time without a reasonable explanation or prior notice.

- Repeatedly missing agreed meetings without notification.
- Failing to update progress or respond to important project information within the agreed period.
- Repeatedly failing to fulfill assigned responsibilities without seeking support or agreeing on an adjustment plan.
- Deliberately submitting work that does not comply with agreed quality standards or academic-integrity requirements.
- Misrepresenting contribution levels or deliberately obstructing another member's work.

A single mistake, a genuine emergency, or a good-faith request for support is not automatically considered a violation.

9.3. Graduated Response Process

When a violation occurs, the team should respond proportionately:

1. **First occurrence:** Discuss the issue privately, clarify expectations, and agree on corrective actions.
2. **Repeated violation:** Record the issue and establish a specific improvement plan with clear responsibilities and deadlines.
3. **Ongoing or serious issue:** Review task assignments, provide appropriate support or redistribute work, and consult the lecturer when necessary.

Serious issues involving academic-integrity violations, harassment, or intentional harm may be escalated immediately.

9.4. Evaluating Individual Contributions

- Each member's contribution will be evaluated using multiple sources of evidence, including completed tasks, task-management history, commits, Pull Requests, code reviews, testing, documentation, meeting participation, and other agreed responsibilities.
- Git activity must not be the sole criterion for evaluating individual contributions because planning, testing, research, documentation, and coordination may not produce many commits.
- Members must update progress regularly and maintain accurate task records.
- If there is a dispute over contribution level, the team must review available evidence and give the relevant member an opportunity to explain their work.
- Any individual-contribution report or proposed adjustment must comply with course assessment rules and be discussed with the lecturer when appropriate.

9.5. Fairness and Evidence-Based Recognition

- Responses must be consistent, proportionate, and based on verifiable evidence.
- Reassigning tasks must be discussed with affected members and should not be used as punishment without considering the circumstances.
- The team may not independently change official course grades or impose penalties beyond its authority. Such matters must follow the lecturer's and course's regulations.

10. Academic Integrity and AI Use

10.1. Academic Integrity

- All members must comply with the university's academic-integrity rules and course requirements.

- Plagiarism, unauthorized collaboration, falsification of results, and presenting another person's work as one's own are strictly prohibited.
- Source code, text, datasets, images, and other external materials must be used under appropriate licenses and acknowledged when necessary.
- Members must not submit work they do not understand or falsely claim that another person's work is their independent work.

10.2. Responsible Use of Generative AI

- Generative AI tools may be used to support learning and productivity, such as brainstorming, explaining concepts, suggesting test cases, reviewing code, improving documentation, or identifying potential defects, provided that course rules are followed.
- AI-generated code, explanations, data, and other content must be reviewed, tested, and verified by members before being included in the project.
- Members remain responsible for the accuracy, security, originality, and appropriateness of submitted work, regardless of whether AI was used.
- AI must not be used to fabricate references, test results, experimental evidence, or project progress.
- Members must not enter confidential information, authentication information, private keys, personal data, or other sensitive information into AI tools without authorization and appropriate safeguards.
- The team must comply with all course-specific restrictions on AI use, including disclosure requirements and limitations for particular assignments or deliverables.

10.3. Transparency and Acknowledgement of AI Use

- When required by the lecturer or course rules, members must disclose the AI tools used, their purpose, and the parts of the work assisted by AI.
- Significant AI-assisted contributions must be acknowledged in the report, appendix, or appropriate project record when required.
- References and citations suggested by AI must be independently verified using reliable sources.
- Members must be able to explain and defend the source code, algorithms, design decisions, and conclusions in their submission.

10.4. Responsibility

- Each member is responsible for ensuring that their contribution complies with academic-integrity requirements.
- If the team is unsure whether a particular use of AI is permitted, it must check assignment instructions or ask the lecturer before proceeding.
- Suspected violations must be handled fairly and in accordance with official course regulations.

11. Amendments to the Team Agreement

11.1. Proposing Amendments

- Any member may propose an amendment when project requirements, team circumstances, or working methods change.
- Proposed amendments must be shared with all members, along with a clear explanation of their reasons and expected impact.

11.2. Discussion and Approval

- The team will discuss proposed amendments and seek consensus.
- If consensus cannot be reached, an amendment may be approved with at least a two-thirds majority of all current team members.
- Members must have sufficient time to review and comment before voting.
- Changes that affect a specific member's responsibilities must be discussed with that member before implementation.
- Amendments must not replace or conflict with university regulations, lecturer guidance, or official course requirements.

11.3. Record Keeping and Effective Date

- Every approved amendment must be recorded in the team agreement, including the amendment date, the changed content, and the approval result.
- The updated agreement must be shared with all members.
- Unless otherwise agreed, amendments take effect after approval and must not be applied retroactively to penalize members for actions that complied with the previous version.

11.4. Periodic Review

- The team should review the agreement at important project milestones or when significant issues arise.
- All members are responsible for following the latest approved version of the team agreement.

12. Signatures

By signing below, all team members confirm that they have read, understood, and agreed to the terms of this Team Contract.

No.	Full Name	Student ID	Signature / Confirmation	Date
1	Nguyễn Hoàng Danh	25127031	Agreed	2026/10/12
2	Lê Minh Trí	25127527	Agreed	2026/10/12
3	Nguyễn Trung Kiên	25127397	Agreed	2026/10/12
4	Trương Tấn Phúc	25127467	Agreed	2026/10/13
5	Hồ Hoàng Tiến Dũng	25127304	Agreed	2026/10/13
6	Phạm Tiến Đạt	25127034	Agreed	2026/10/13
7	Nguyễn Chí Vĩ	25127549	Agreed	2026/10/13

Appendix

A.AI Usage Notes

- Tool name: Chat GPT (verison: GPT-6, platform: microsoft edge)
- Access time: 19:20 (2026/10/12)

- Prompts used: Quy định cách thức sửa đổi thỏa thuận nhóm và tỷ lệ thành viên cần đồng ý để thông qua một thay đổi. Hãy đóng vai trò là một giảng viên về khoa học máy tính, đã có kinh nghiệm trong làm việc theo nhóm của các công ty lớn nước ngoài. Hãy đề xuất cho tôi mọt bản team contract từ nội dung 5 đến 11
- Purpose of use: Proposed Content for Sections 5–11
- Which content was generated by AI: Sections 5-11
- Which content was done independently: section 6.
- How my goupmates and I edited or validated it: through **prompt1**: Tôi sẽ nói một quy trình gitflow như thé này, bạn xem chỗ nào không hợp lý thì nói cho tôi. Tất cả thành viên sẽ được add vô repo. Code hoạt động tốt sẽ ở nhánh main. Các nhánh feature khi làm xong, phải rebase ngay trên local repo của người đó sau đó mới push lên nhánh feature và tạo pull request để đợi người review code xem và duyệt sau đó merge vào main.**Prompt2**: **Ở bước 4: chỉ pull nhánh main về, sau đó rebase. - Rebase: có nghĩa là tái cơ sở lại. Ở nhánh main, bạn cập nhật những thay đổi mới nhất của nó, sau đó quay lại nhánh feature , dùng câu lệnh git rebase main, để gộp nhánh main và nhánh feature của bạn lại.(lí do là để test tính năng của bạn có chạy được trên main khong) .Sau khi đã chạy được, push nhánh tính năng lên trên central repo(lưu ý chỉ push nhánh feature thôi). Sau đó lên github tạo pull request để reviewer check và chính reviewer sẽ là người merge code của bạn mới main.
  ***LƯU Ý:** Sẽ có rất nhiều pull request được tạo, vì vậy nếu như lúc đầu bạn đã cập nhật main , nhưng tôi xủ lý và merge code của pull request trước đó thì tôi sẽ yêu cầu bạn rebase lại . hợp lý chưa
- Screenshots or chat history: https://chatgpt.com/share/6ad04468-52dc-83ec-8541-16799228dc3c, https://chatgpt.com/share/6ad04499-6810-83ec-9001-622a90098e80